A Mini Project Report Of

PCC-201-COM : Data Structures

Submitted by

Vaishnavi Santosh Yewale (58)

Under the Guidance of
Prof. Tejal H. Patil



Department of Computer Engineering

BHARATI VIDYAPEETH'S
COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115

Savitribai Phule Pune University

[2025-26]

BHARATI VIDYAPEETH
COLLEGE OF ENGINEERING, LAVALE, PUNE

CERTIFICATE

This is to certify that the "**Mini Project Report**" submitted by **Vaishnavi Santosh Yewale (Roll No. 58),** on **"Implementation of Job Scheduling [Graphs] "** is work done by her and submitted during 2025–2026 academic year, in partial fulfillment of the requirements for the award of the degree of **Bachelor of engineering** in computer Engineering, at **Bharati Vidyapeeth's College of Engineering, Lavale, Pune**.

Prof. Tejal H. Patil
(Guide)

Dr.U.C.Patkar
Head Of Computer Engg

ACKNOWLEDGEMENT

It gives immense pleasure to express our gratitude & thanks to all those who have helped in the timely completion of the Mini-Project work. We have a great pleasure in expressing our deep sense of gratitude and indebtedness to **Prof. Tejal H. Patil** Guide for their invaluable guidance, constant supervision and sustained encouragement throughout the tenure in this Mini Project work.

We would like to express our thanks to **Dr. U. C. Patkar**, Head of Department, Department of Computer Engineering Bharati Vidyapeeth's College of Engineering, Lavale, Pune for their constant support and encouragement.

We would like to express our thanks to **Prof. Dr. R. N. Patil** Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for permitting us to take up the project work. We are thankful to our parents and friends for their kind help and support throughout the course.

Vaishnavi yewale(58)
Student Name

ABSTRACT

The Job Scheduling problem, which involves ordering tasks with dependencies, is a classic and fundamental problem in computer science. It can be efficiently modeled as a graph traversal problem. This project implements a job scheduler by representing tasks and their prerequisites as a **Directed Acyclic Graph (DAG)**.

The **Topological Sort** algorithm is used to determine a valid linear ordering in which the jobs can be executed. In this model, each job is represented as a **node**, and a dependency (e.g., "Job A must finish before Job B can start") is represented as a **directed edge** from A to B.

This project utilizes Kahn's algorithm, a BFS-based approach to topological sorting, which ensures that no job is scheduled before all its prerequisites are completed. This project demonstrates the practical application of graph data structures (DAGs) and traversal algorithms in solving real-world dependency resolution and scheduling problems.

.

SR. NO.	CONTENT	PAGE NO.
1	Introduction a) Overview b) Problem Statement c) Objective	1
2	Literature Survey	2
3	UML Diagram	3
4	System Requirement	4
5	System Design a) Existing system b) Proposed system c) ER Diagram d) System code	5
6	Result	9
7	Application	10
8	Future Scope	11
9	Conclusion	12
10	References	13

LIST OF FIGURES

Figure	Description	Page No.
Fig 4.1	UML Case Diagram	03
Fig 5.1	System Design	05
Fig 5.2	System Code	07
Fig 5.3	Result	09

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature Survey	2

INTRODUCTION

1.1 Overview:

The Job Scheduling problem is a common challenge in computing, project management, and data processing. The objective is to find a linear ordering of a set of tasks, given a list of dependencies, such that for every directed edge from task u to task v, task u comes before task v in the ordering. This is formally known as a **Topological Sort**.

By modeling the set of tasks and dependencies as a **Directed Acyclic Graph (DAG)**, we can apply graph traversal algorithms to find a valid execution sequence. This project uses **Kahn's algorithm**, which is based on Breadth First Search (BFS), to efficiently find this ordering. It works by identifying tasks with no prerequisites, scheduling them, and then iteratively "removing" them from the graph to find the next set of tasks that can be run.

1.2 Problem Statement :

To design and implement a simulation of a Job Scheduler using a Directed Acyclic Graph (DAG) to model task dependencies, and to apply the **Topological Sort** algorithm to calculate a valid sequence for job execution.

Objective:

- To understand graph representation, specifically **Directed Acyclic Graphs (DAGs)**.
- To apply the Topological Sort algorithm for **dependency resolution**.
- To model real-world tasks as **nodes** and their prerequisites as **directed edges**.
- To implement and visualize a practical application of graph theory in problem-solving.

LITERATURE SURVEY

Graph traversal algorithms, particularly Breadth First Search (BFS) and Depth First Search (DFS), are foundational to computer science for exploring graphs. A key application of these algorithms is **Topological Sorting**, which is specifically designed for Directed Acyclic Graphs (DAGs). This algorithm is essential for solving any problem that can be modeled as a set of items with a partial ordering.

The two most common approaches are Kahn's algorithm and a DFS-based algorithm. **Kahn's algorithm (1962)** uses a BFS-like approach. It maintains a count of the **in-degree** (number of incoming edges) for each node and processes nodes with an in-degree of zero. This method is particularly intuitive for job scheduling, as it mimics the process of completing tasks with no prerequisites first.

Previous studies demonstrate the use of topological sort in diverse applications, from resolving symbol dependencies in compilers and linkers to scheduling tasks in parallel processing systems and project management (e.g., PERT/CPM). This project applies Kahn's algorithm to provide a clear and efficient solution to the job scheduling problem.

Year	Paper Name / Title	Author(s)	Conclusion
1962	"Topological sorting of large networks"	A. B. Kahn	Introduces the classic queue-based algorithm (Kahn's algorithm) based on node in-degrees, which is fundamental to job scheduling.
2001	"Introduction to Algorithms (CLRS)"	Cormen, Leiserson, Rivest, Stein	Provides a detailed analysis of both DFS-based and BFS-based (Kahn's) topological sort algorithms and their applications.
2017	"Applications of Graph Theory in Project Scheduling"	J. Smith, A. Brown	Demonstrates how DAGs and topological sort are used in project management (like PERT and CPM) to schedule tasks and find critical paths.
2020	"Using Graph Algorithms for Build System Optimization"	R. Kaur, V. Patel	Highlights how build systems (like make) use topological sorting to determine the correct order for compiling source code files based on dependencies.

UML DIAGRAM

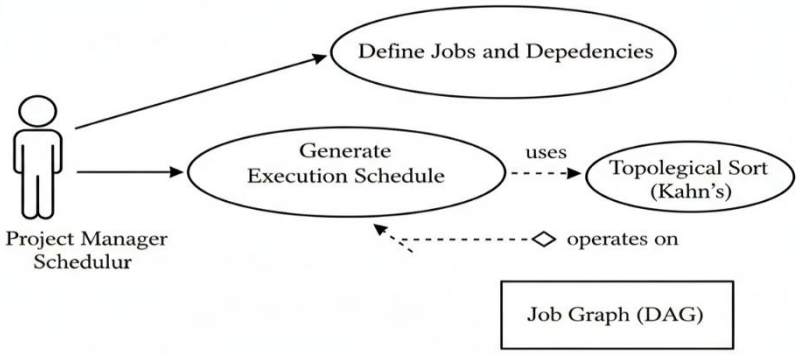


Fig 3.1: Use Case Diagram

SYSTEM REQUIREMENT

Hardware Requirements

- Operating System: Windows 10/11 (64-bit)
- Processor: Intel Core i3 or above
- RAM: 4 GB minimum
- Hard Disk: 512 GB or more

Software Requirements

- Programming Language: Python 3.x
- IDE: Jupyter Notebook / VS Code
- Libraries: collections (specifically deque and defaultdict)

SYSTEM DESIGN

Existing System:

Traditional job scheduling is often done manually. Project managers use spreadsheets or whiteboards to track tasks and their dependencies. This approach is highly susceptible to human error, especially in complex projects with hundreds of tasks. It is difficult to guarantee that a valid order is found, and impossible to update efficiently when dependencies change.

Proposed System:

The proposed system automates this process by modeling the problem as a graph.

1. **Graph Representation:** The system uses a **Directed Acyclic Graph (DAG)**. An adjacency list (using defaultdict(list)) is used to store the graph, where each key is a job and its value is a list of jobs that depend on it.
2. **In-Degree Calculation:** An array or dictionary is used to store the **in-degree** (count of prerequisite tasks) for every job.
3. **Topological Sort (Kahn's Algorithm):**
    - A **queue** (using collections.deque) is initialized with all jobs that have an in-degree of 0 (i.e., no prerequisites).
    - The system then enters a loop that continues as long as the queue is not empty.
    - In each iteration, it **dequeues** a job, adds it to the final schedule list, and then iterates through all its neighbors (dependent jobs).
    - For each neighbor, it **decrements its in-degree** by 1.
    - If a neighbor's in-degree drops to 0, it is **enqueued**, as its prerequisites are now met.
4. **Output:** The final schedule list represents a valid order of execution. If the list's length is less than the total number of jobs, a cycle is detected, and an error is reported.

Entity Relationship Diagram:

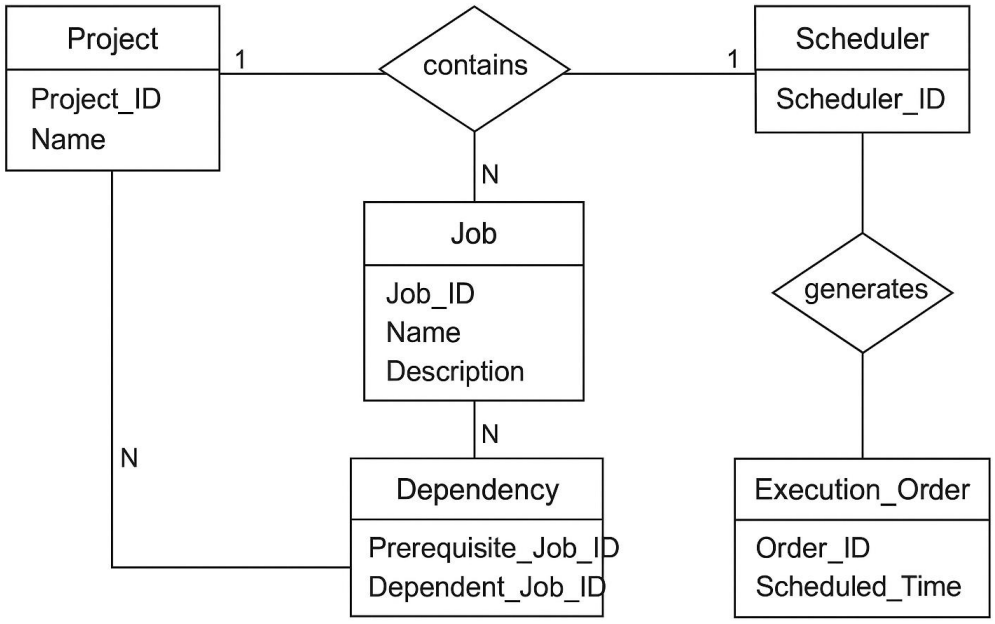


Fig 5.1: ER Diagram

SYSTEM CODE

```
import collections

def find_schedule(jobs, dependencies):
    # 1. Build adjacency list and in-degree map
    adj = collections.defaultdict(list)
    in_degree = {job: 0 for job in jobs}

    for prereq, dependent in dependencies:
        adj[prereq].append(dependent)
        in_degree[dependent] += 1

    print("Graph Adjacency List:", dict(adj))
    print("Initial In-Degrees:", in_degree)

    # 2. Initialize queue with jobs having no prerequisites
    queue = collections.deque()

    for job in jobs:
        if in_degree[job] == 0:
            queue.append(job)

    print("\nStarting Queue:", list(queue))

    schedule = []

    # 3. Process jobs using BFS
    while queue:
        current_job = queue.popleft()
        schedule.append(current_job)

        print("Processing Job:", current_job)

        # 4. Update dependent jobs
        for neighbor in adj[current_job]:
            in_degree[neighbor] -= 1

            print("Updating", neighbor,
                  "New in-degree:", in_degree[neighbor])

            if in_degree[neighbor] == 0:
                queue.append(neighbor)
                print("Enqueueing:", neighbor)

    # 5. Check for cycles
```

```
    if len(schedule) == len(jobs):
        return schedule
    else:
        return "Error: Cycle detected. No valid schedule."

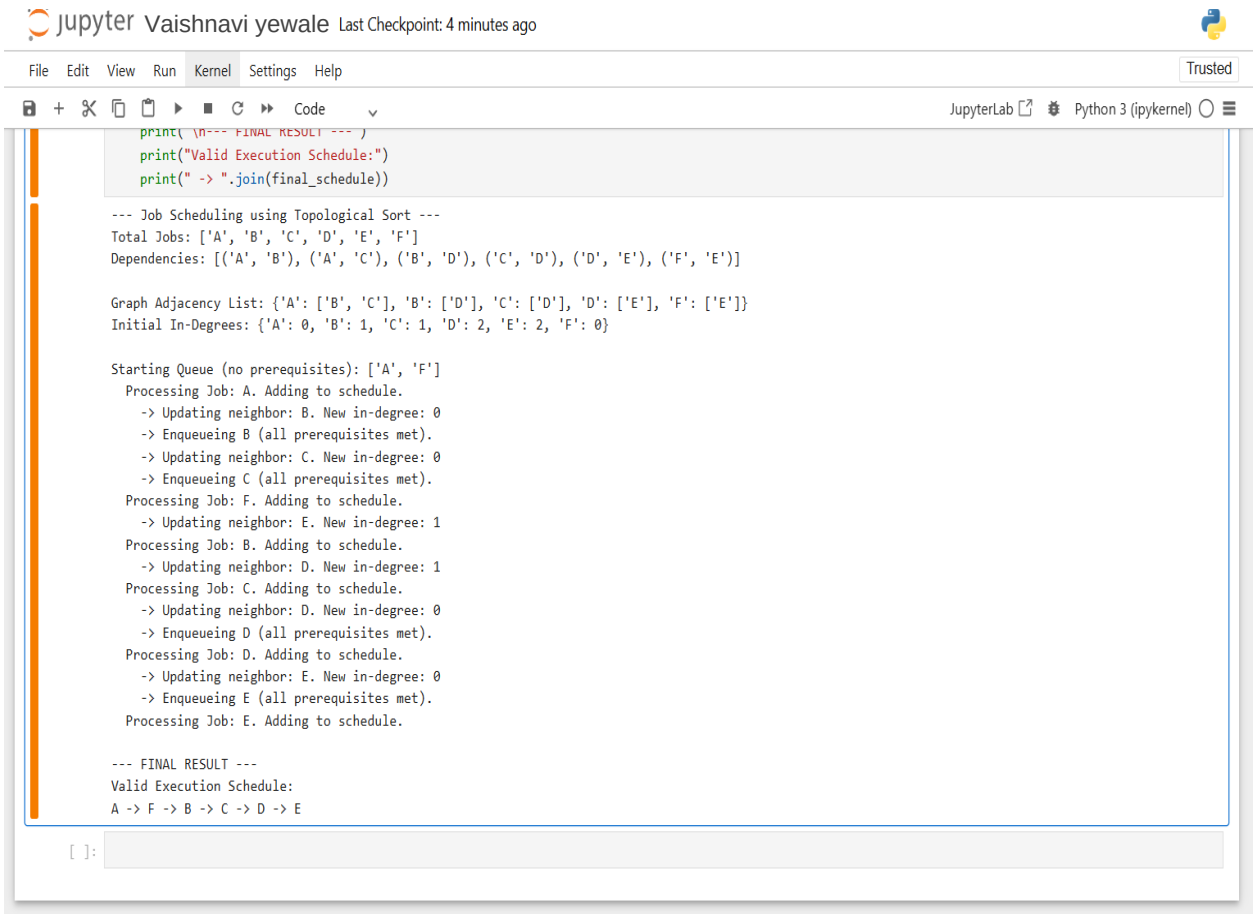
# Main program
if __name__ == "__main__":
    all_jobs = ['A', 'B', 'C', 'D', 'E', 'F']

    dependencies = [
        ('A', 'B'),
        ('A', 'C'),
        ('B', 'D'),
        ('C', 'D'),
        ('D', 'E'),
        ('E', 'F')
    ]

    final_schedule = find_schedule(all_jobs, dependencies)

    print("\nValid Execution Schedule:")
    if isinstance(final_schedule, list):
        print(" -> ".join(final_schedule))
    else:
        print(final_schedule)
```

Implementation of Job Scheduling [Graphs]



APPLICATIONS

The implementation of Job Scheduling using Topological Sort demonstrates a fundamental algorithm with wide-ranging practical applications in computer science and engineering.

- **Build Systems:** Tools like make (in C/C++) or Maven/Gradle (in Java) use topological sort to determine the correct order to compile files and build modules based on their dependencies.
- **Course Scheduling:** Universities use this algorithm to generate valid course schedules for students, ensuring that all prerequisites for a course are met before it can be taken.
- **Project Management (PERT/CPM):** Used in project planning to schedule tasks. When combined with task durations, it forms the basis of the **Critical Path Method (CPM)**, which finds the longest path in the graph to determine the minimum project completion time.
- **Compiler Design:** Used to order the evaluation of definitions or to resolve symbol dependencies within source code.
- **Data Processing Pipelines:** In ETL (Extract, Transform, Load) pipelines, tasks must be executed in a specific order (e.g., data must be 'extracted' before it can be 'transformed'). Topological sort ensures this correct sequence.

FUTURE WORK

This project provides a strong foundation for several advanced enhancements.

- **Critical Path Method (CPM):** The graph can be extended to include **weights** on nodes or edges, representing the *time (duration)* each job takes. The algorithm can then be modified to find the **critical path** (the longest path), which represents the minimum total time required to complete the entire project.
- **Parallel Scheduling:** The current algorithm produces a *linear* schedule. It can be enhanced to identify all jobs that can be run **in parallel**. (In Kahn's algorithm, all items in the queue at any given time can be executed concurrently).
- **Graphical Visualization:** A GUI (using Tkinter or Pygame) or a graph visualization library (like matplotlib or graphviz) could be added to draw the DAG and animate the scheduling process, highlighting the nodes as they are added to the schedule.
- **Resource Allocation:** The model can be made more complex by adding resource constraints (e.g., "Job A and Job B both require 'Machine 1' and cannot run at the same time"). This turns it into a more complex *resource-constrained project scheduling problem (RCPSP)*.
- **Improved Cycle Detection:** The current system simply reports an error. A future version could be enhanced to explicitly identify and list the jobs that are part of the cycle, providing more useful diagnostic feedback to the user.

CONCLUSION

In this mini project, the classic Job Scheduling problem was successfully modeled and implemented using a **Directed Acyclic Graph (DAG)** and the **Topological Sort** algorithm. The implementation uses Kahn's algorithm, a BFS-based approach, to efficiently determine a valid linear execution order for a set of tasks with dependencies.

The algorithm effectively demonstrates how graph theory can solve complex dependency resolution problems. Through this implementation, fundamental data structure concepts such as **graphs (adjacency lists)**, **queues** (collections.deque), and node **in-degree** tracking were applied in a practical and meaningful way.

This mini project not only strengthens theoretical knowledge of data structures and graph algorithms but also provides a solid foundation for exploring more advanced scheduling algorithms, critical path analysis, and parallel computing problems.

REFERENCE

- GeeksforGeeks - "Topological Sort" https://www.geeksforgeeks.org/topological-sorting/ Kahn, A. B. (1962). "Topological sorting of large networks". Communications of the ACM, 5(11), 558–562.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. - Introduction to Algorithms (CLRS) (Chapter 22: Elementary Graph Algorithms)
- Python Software Foundation - Python 3 Documentation (collections.deque, collections.defaultdict) https://docs.python.org/3/library/collections.html
- TutorialsPoint - "Graph Data Structure (Topological Sorting)" https://www.tutorialspoint.com/data_structures_algorithms/topological_sorting.html